

Moving Average Strategy creation and Backtesting

STEP 1 : Coming up with a trading idea (Using 5 min data)

we determine the 12 period simple moving average and compare it with the price at that time .

There are two conditions which we check:

1. If the Price is greater than SMA 12 , we go long . We continue to stay invested untill the square-off condition is satisfied.
2. When the price becomes less than the SMA 12 , we square off our long position

Our trading rules can be stated as:

1. "BUY" when Price > SMA 12
2. SQUARE off when price <= SMA12

Prerequisites 1

```
In [3]: import pandas as pd
import numpy as np
import datetime ## is to know the date of our choice
import yfinance as yf ## is to download the data from yahoofinance
import matplotlib.pyplot as plt ## is to plot the graph
import warnings
warnings.filterwarnings("ignore")
```

```
In [5]: end1 = datetime.datetime.today().date()
end1
```

```
Out[5]: datetime.date(2026, 9, 14)
```

```
In [10]: start1 = end1 - pd.Timedelta(days=59)
start1
```

```
Out[10]: datetime.date(2026, 7, 17)
```

```
In [11]: df = yf.download("NVDA", start=start1, end = end1, interval = "5m", auto_adjust = Fa
df
```

```
[*****100%*****] 1 of 1 completed
```

Out[11]:

Price	Adj Close	Close	High	Low	Open	Volume
Ticker	NVDA	NVDA	NVDA	NVDA	NVDA	NVDA
Datetime						
2026-07-17 13:30:00+00:00	200.225006	200.225006	202.735001	199.940002	202.735001	17223231
2026-07-17 13:35:00+00:00	199.429993	199.429993	200.639999	199.410004	200.210007	4885672
2026-07-17 13:40:00+00:00	198.345001	198.345001	199.979996	197.970001	199.429993	4891151
2026-07-17 13:45:00+00:00	199.244995	199.244995	199.610001	198.259995	198.330994	3128718
2026-07-17 13:50:00+00:00	200.539993	200.539993	200.559998	199.029999	199.229996	2628981
...	...	...	...	...	...	...
2026-09-11 19:35:00+00:00	218.470093	218.470093	218.550003	218.380096	218.460007	751964
2026-09-11 19:40:00+00:00	218.765793	218.765793	218.839996	218.410004	218.485001	1052449
2026-09-11 19:45:00+00:00	218.759995	218.759995	218.850006	218.630005	218.759995	726786
2026-09-11 19:50:00+00:00	218.720001	218.720001	218.824997	218.399994	218.750000	1536086
2026-09-11 19:55:00+00:00	218.199997	218.199997	218.789993	218.149994	218.679993	2753764

3120 rows × 6 columns

In [12]:
import os

In [13]:
os.getcwd()

Out[13]:
'C:\\Users\\Punith Kumar G L'

In [14]:
DMP_01_data_NVDA_5min_interval

In [32]:
df = pd.read_csv("DMP_01_data_NVDA_5min_interval.csv", index_col= "Datetime" , parse_dates=True)
df1 = df.copy()

In [33]:
df1.drop(["Open" , "High" , "Volume", "Low"] , axis=1 , inplace=True)
df1

Out[33]:

	Close	Adj Close
Datetime		
2025-03-31 13:30:00+00:00	104.489998	104.489998
2025-03-31 13:35:00+00:00	104.419998	104.419998
2025-03-31 13:40:00+00:00	103.848999	103.848999
2025-03-31 13:45:00+00:00	104.180000	104.180000
2025-03-31 13:50:00+00:00	104.675003	104.675003
...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993
2025-05-28 19:40:00+00:00	135.554993	135.554993
2025-05-28 19:45:00+00:00	135.650208	135.650208
2025-05-28 19:50:00+00:00	135.226898	135.226898
2025-05-28 19:55:00+00:00	134.811600	134.811600

3196 rows × 2 columns

```
In [70]: df1["cc_returns"] = df1["Close"].pct_change()  
df1
```

Out[70]:

	Close	Adj Close	cc_returns	SMA	Position	Strategyreturns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	NaN	NaN	NaN	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	-0.000670	NaN	NaN	NaN
2025-03-31 13:40:00+00:00	103.848999	103.848999	-0.005468	NaN	NaN	1.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	0.003187	NaN	0.0	1.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	0.004751	NaN	0.0	1.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	-0.003296	136.768266	0.0	1.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	-0.003565	136.652431	0.0	1.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	0.000702	136.554115	0.0	1.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	-0.003121	136.446774	0.0	1.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	-0.003071	136.272615	0.0	1.0

3196 rows × 6 columns

In [71]:

```
sma = 12
df1["SMA"] = df1["Close"].rolling(window=sma).mean()
df1
```

Out[71]:

	Close	Adj Close	cc_returns	SMA	Position	Strategyreturns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	NaN	NaN	NaN	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	-0.000670	NaN	NaN	NaN
2025-03-31 13:40:00+00:00	103.848999	103.848999	-0.005468	NaN	NaN	1.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	0.003187	NaN	0.0	1.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	0.004751	NaN	0.0	1.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	-0.003296	136.768266	0.0	1.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	-0.003565	136.652431	0.0	1.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	0.000702	136.554115	0.0	1.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	-0.003121	136.446774	0.0	1.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	-0.003071	136.272615	0.0	1.0

3196 rows × 6 columns

```
In [72]: df1["Position"] = np.where((df1["Close"] > df1["SMA"]), 1,0)
df1
```

Out[72]:

	Close	Adj Close	cc_returns	SMA	Position	Strategyreturns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	NaN	NaN	0	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	-0.000670	NaN	0	NaN
2025-03-31 13:40:00+00:00	103.848999	103.848999	-0.005468	NaN	0	1.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	0.003187	NaN	0	1.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	0.004751	NaN	0	1.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	-0.003296	136.768266	0	1.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	-0.003565	136.652431	0	1.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	0.000702	136.554115	0	1.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	-0.003121	136.446774	0	1.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	-0.003071	136.272615	0	1.0

3196 rows × 6 columns

In [73]: df1["Position"] = df1["Position"].shift(1)
df1

Out[73]:

	Close	Adj Close	cc_returns	SMA	Position	Strategyreturns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	NaN	NaN	NaN	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	-0.000670	NaN	0.0	NaN
2025-03-31 13:40:00+00:00	103.848999	103.848999	-0.005468	NaN	0.0	1.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	0.003187	NaN	0.0	1.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	0.004751	NaN	0.0	1.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	-0.003296	136.768266	0.0	1.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	-0.003565	136.652431	0.0	1.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	0.000702	136.554115	0.0	1.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	-0.003121	136.446774	0.0	1.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	-0.003071	136.272615	0.0	1.0

3196 rows × 6 columns

In [74]:

```
df1["Strategyreturns"] = df1["cc_returns"]*df1["Position"]
df1
```

Out[74]:

	Close	Adj Close	cc_returns	SMA	Position	Strategyreturns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	NaN	NaN	NaN	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	-0.000670	NaN	0.0	-0.0
2025-03-31 13:40:00+00:00	103.848999	103.848999	-0.005468	NaN	0.0	-0.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	0.003187	NaN	0.0	0.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	0.004751	NaN	0.0	0.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	-0.003296	136.768266	0.0	-0.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	-0.003565	136.652431	0.0	-0.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	0.000702	136.554115	0.0	0.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	-0.003121	136.446774	0.0	-0.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	-0.003071	136.272615	0.0	-0.0

3196 rows × 6 columns

```
In [75]: df1["cc_returns"] = 1 + df1["cc_returns"]

In [76]: df1["Strategyreturns"] = 1 + df1["Strategyreturns"]

In [77]: df1
```

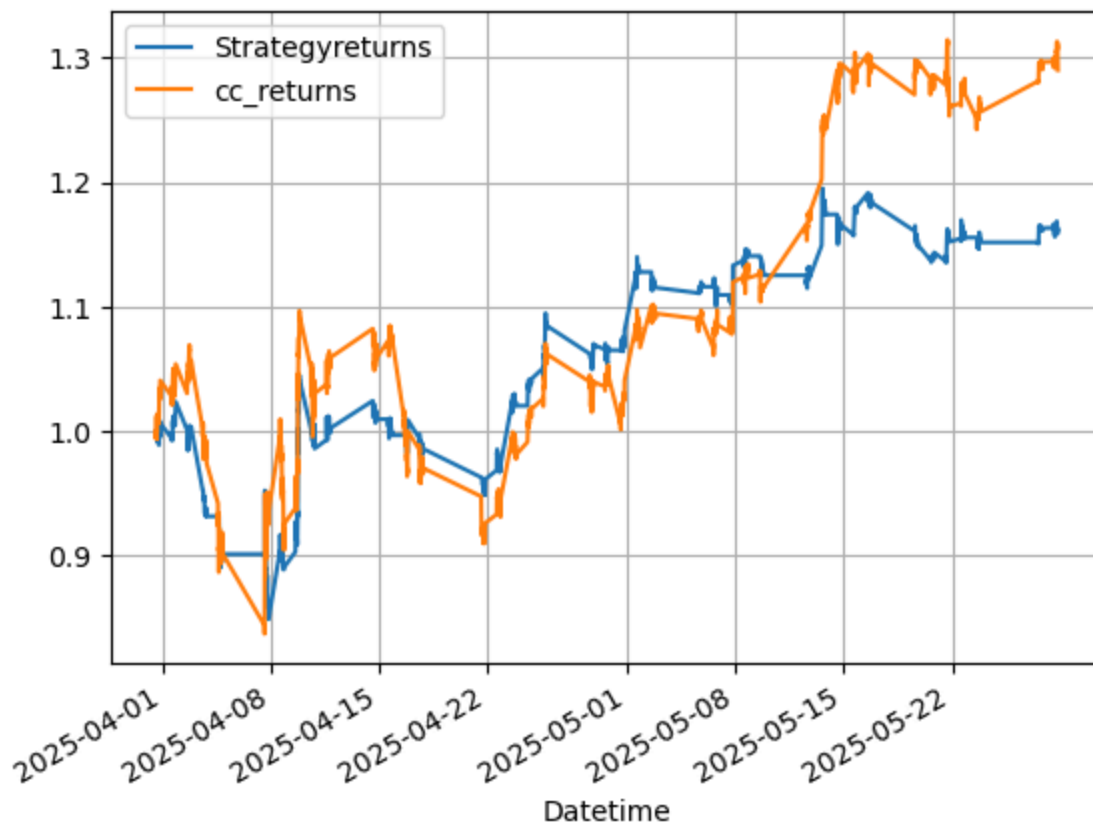
Out[77]:

	Close	Adj Close	cc_returns	SMA	Position	Strategyreturns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	NaN	NaN	NaN	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	0.999330	NaN	0.0	1.0
2025-03-31 13:40:00+00:00	103.848999	103.848999	0.994532	NaN	0.0	1.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	1.003187	NaN	0.0	1.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	1.004751	NaN	0.0	1.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	0.996704	136.768266	0.0	1.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	0.996435	136.652431	0.0	1.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	1.000702	136.554115	0.0	1.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	0.996879	136.446774	0.0	1.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	0.996929	136.272615	0.0	1.0

3196 rows × 6 columns

In [79]: df1[["Strategyreturns" , "cc_returns"]].cumprod().plot(grid = True)

Out[79]: <Axes: xlabel='Datetime'>



```
In [84]: print("Buy and Hold :", np.round(df1["cc_returns"].cumprod()[-1]-1,2))
print("Strategyreturns:", np.round(df1["Strategyreturns"].cumprod()[-1]-1,2))
```

Buy and Hold : 0.29
Strategyreturns: 0.16

In []:

Exponential Moving Average (Long Only)

EMA = assigning weights to the data

```
In [87]: k = df.copy()
k
```

Out[87]:

Datetime	Open	High	Low	Close	Adj Close	Volume
2025-03-31 13:30:00+00:00	105.110001	105.789902	104.300003	104.489998	104.489998	26407346
2025-03-31 13:35:00+00:00	104.489998	105.169998	104.260002	104.419998	104.419998	8501363
2025-03-31 13:40:00+00:00	104.410004	104.480003	103.750000	103.848999	103.848999	7910612
2025-03-31 13:45:00+00:00	103.839996	104.349998	103.650002	104.180000	104.180000	5674473
2025-03-31 13:50:00+00:00	104.150002	104.730003	104.000000	104.675003	104.675003	5396151
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.484406	136.490005	136.024994	136.039993	136.039993	4094840
2025-05-28 19:40:00+00:00	136.039993	136.070007	135.460007	135.554993	135.554993	5880672
2025-05-28 19:45:00+00:00	135.558395	136.005005	135.460007	135.650208	135.650208	5215595
2025-05-28 19:50:00+00:00	135.660004	135.660004	134.839996	135.226898	135.226898	6378419
2025-05-28 19:55:00+00:00	135.210007	135.350006	134.789993	134.811600	134.811600	9753035

3196 rows × 6 columns

In [88]:

```
k.drop(["Open", "High" , "Low", "Volume"] , axis=1 , inplace=True)
k
```

Out[88]:

	Close	Adj Close
Datetime		
2025-03-31 13:30:00+00:00	104.489998	104.489998
2025-03-31 13:35:00+00:00	104.419998	104.419998
2025-03-31 13:40:00+00:00	103.848999	103.848999
2025-03-31 13:45:00+00:00	104.180000	104.180000
2025-03-31 13:50:00+00:00	104.675003	104.675003
...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993
2025-05-28 19:40:00+00:00	135.554993	135.554993
2025-05-28 19:45:00+00:00	135.650208	135.650208
2025-05-28 19:50:00+00:00	135.226898	135.226898
2025-05-28 19:55:00+00:00	134.811600	134.811600

3196 rows × 2 columns

```

In [94]: Ema = 12
k["EMA"] = k["Close"].ewm(span=Ema, adjust=False).mean()
k["cc_returns"] = k["Close"].pct_change()
k["Position"] = np.where((k["Close"] > k["EMA"]),1,0)
k["Position"] = k["Position"].shift(2)
k["Strategy_returns"] = k["cc_returns"] * k["Position"]
k["Strategy_returns"] = 1+ k["Strategy_returns"]
k["cc_returns"] = 1+k["cc_returns"]
print("Buy and Hold :", np.round(k["cc_returns"].cumprod()[-1]-1,2))
print("Strategyreturns:", np.round(k["Strategy_returns"].cumprod()[-1]-1,2))
k

```

Buy and Hold : 0.29

Strategyreturns: 0.22

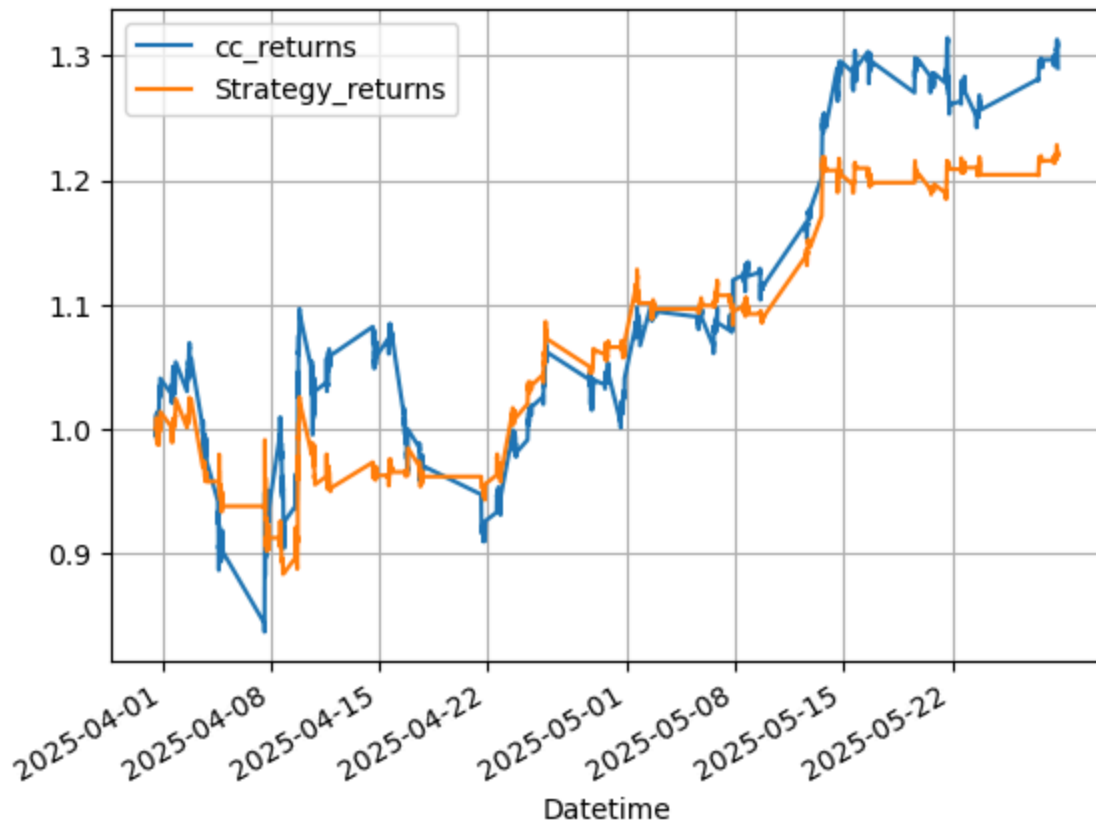
Out[94]:

	Close	Adj Close	EMA	cc_returns	Position	Strategy_returns
Datetime						
2025-03-31 13:30:00+00:00	104.489998	104.489998	104.489998	NaN	NaN	NaN
2025-03-31 13:35:00+00:00	104.419998	104.419998	104.479229	0.999330	NaN	NaN
2025-03-31 13:40:00+00:00	103.848999	103.848999	104.382270	0.994532	0.0	1.0
2025-03-31 13:45:00+00:00	104.180000	104.180000	104.351152	1.003187	0.0	1.0
2025-03-31 13:50:00+00:00	104.675003	104.675003	104.400975	1.004751	0.0	1.0
...	...	...	...	...	...	...
2025-05-28 19:35:00+00:00	136.039993	136.039993	136.665718	0.996704	0.0	1.0
2025-05-28 19:40:00+00:00	135.554993	135.554993	136.494838	0.996435	0.0	1.0
2025-05-28 19:45:00+00:00	135.650208	135.650208	136.364894	1.000702	0.0	1.0
2025-05-28 19:50:00+00:00	135.226898	135.226898	136.189818	0.996879	0.0	1.0
2025-05-28 19:55:00+00:00	134.811600	134.811600	135.977785	0.996929	0.0	1.0

3196 rows × 6 columns

In [96]: k[["cc_returns", "Strategy_returns"]].cumprod().plot(grid=True)

Out[96]: <Axes: xlabel='Datetime'>



In []: